

unit I

The Object Model

AGENDA

- Foundations of the Object Model
- Object-Oriented Programming
- Object-Oriented Design,
- Object- Oriented Analysis
- Elements of the Object Model

Foundations of the Object Model

The header features a dark purple background with a wavy bottom edge. On the right side, there are decorative elements: a large light blue circle, a smaller medium blue circle below it, and a vertical red rectangle at the top right corner.

OBJECT ORIENTED PROGRAMMING

Object-oriented programming is a method of implementation in which programs are organized as cooperative collections of objects, each of which represents an instance of some class, and whose classes are all members of a hierarchy of classes united via inheritance relationships.

OBJECT ORIENTED PROGRAMMING

There are three important parts to this definition:

- (1) Object-oriented programming uses objects, not algorithms, as its fundamental logical building blocks**
- (2) each object is an instance of some class; and**
- (3) classes may be related to one another via inheritance relationships**

OBJECT ORIENTED DESIGN

- The emphasis in programming methods is primarily on the proper and effective use of particular language mechanisms.
- By contrast, design methods emphasize the proper and effective structuring of a complex system.

OBJECT ORIENTED DESIGN

Object-oriented design is a method of design encompassing the process of object-oriented decomposition and a notation for depicting both logical and physical as well as static and dynamic models of the system under design.

OBJECT ORIENTED DESIGN

There are two important parts to this definition:
object-oriented design

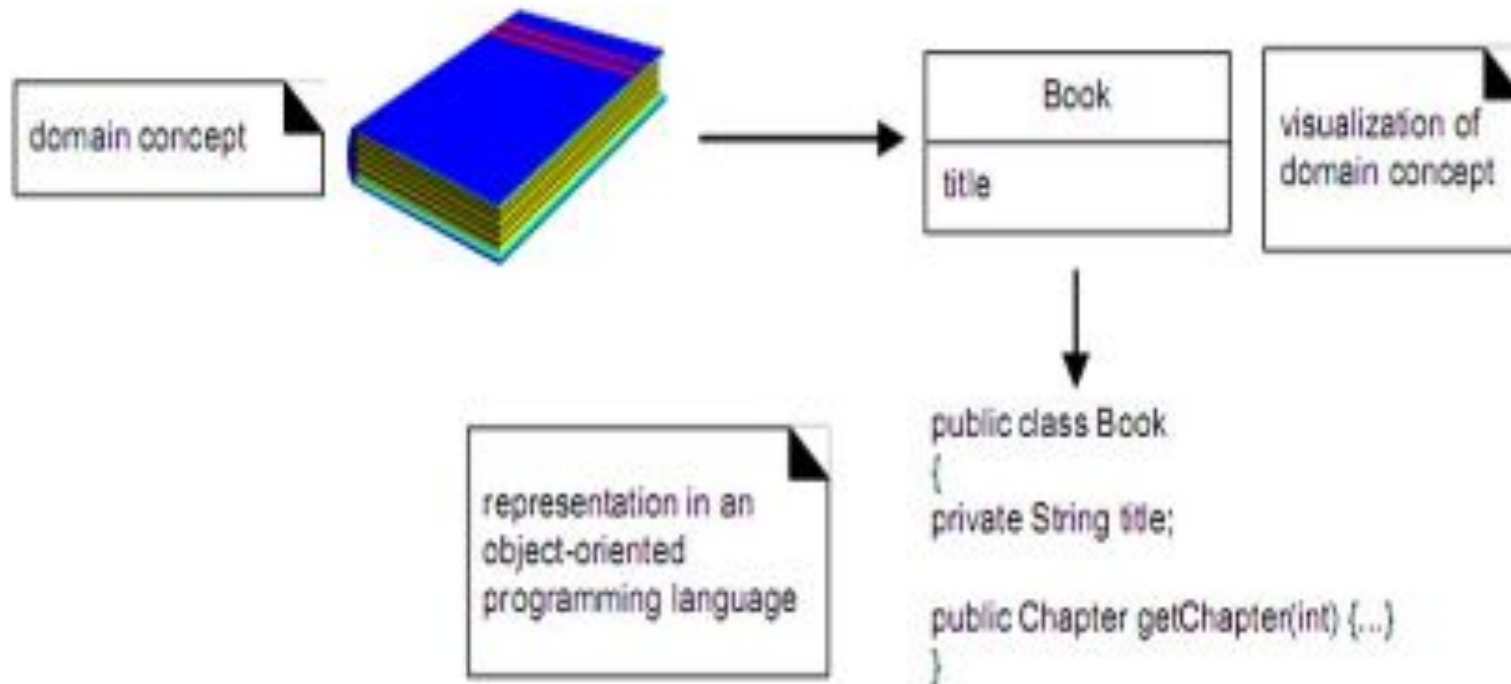
- (1) Leads to an object-oriented decomposition and
- (2) Uses different notations to express
different models of the logical (class and object structure)
and physical (module and process architecture) design of a
system, in addition to the static and dynamic aspects of the
system.

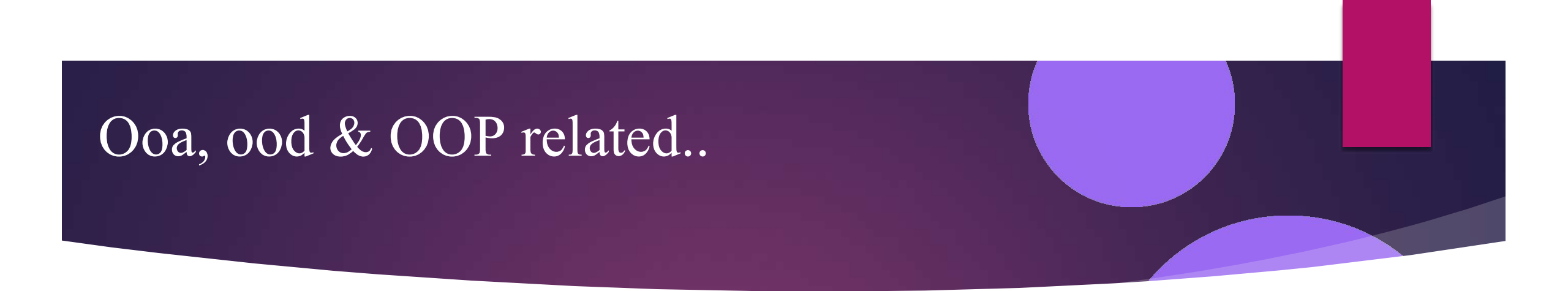
OBJECT ORIENTED analysis

□ Object-oriented analysis (OOA) emphasizes the building of real-world models, using an object-oriented view of the world

□ Object-oriented analysis is a method of analysis that examines requirements from the perspective of the classes and objects found in the vocabulary of the problem domain.

Ooa, ood & OOP related..





Ooa, ood & OOP related..

□ The products of **object-oriented analysis** serve as the models from which we may start an **object-oriented design**

□ The products of **object-oriented design** can then be used as blueprints for completely implementing a system using **object-oriented programming** methods.

Elements of the Object Model

□ There are five main kinds of programming styles, listed here with the kinds of abstractions they employ:

1. Procedure-oriented - Algorithms
2. Object-oriented - Classes and objects
3. Logic-oriented - Goals, often expressed in a predicate calculus
4. Rule-oriented - If–then rules
5. Constraint-oriented - Invariant relationships

Elements of the Object Model

- Rule-oriented programming would be best suited for the design of a knowledge base
- Procedure-oriented programming would be best for the design of computation-intensive operations.
- The object-oriented style is best suited to the broadest set of applications; indeed, this programming paradigm often serves as the architectural framework in which we employ other paradigms.

Elements of the Object Model

□ For Object-oriented, the conceptual framework is the object model.

□ There are four major elements of this model:

1. Abstraction
2. Encapsulation
3. Modularity
4. Hierarchy

By **major**, we mean that a model without any one of these elements is not object-oriented.

Elements of the Object Model

□ There are three minor elements of the object model:

1. Typing
2. Concurrency
3. Persistence

□ By **minor**, we mean that each of these elements is a useful, but not essential, part of the object model.

abstraction

- Abstraction arises from a recognition of similarities between certain objects, situations, or processes in the real world, and the decision to concentrate upon these similarities and to ignore for the time being the differences
- We can also define an abstraction as a simplified description, or specification, of a system that emphasizes some of the system's details or properties while suppressing others.
- A good abstraction is one that emphasizes details that are significant to the reader or user and suppresses details that are, at least for the moment, immaterial or diversionary

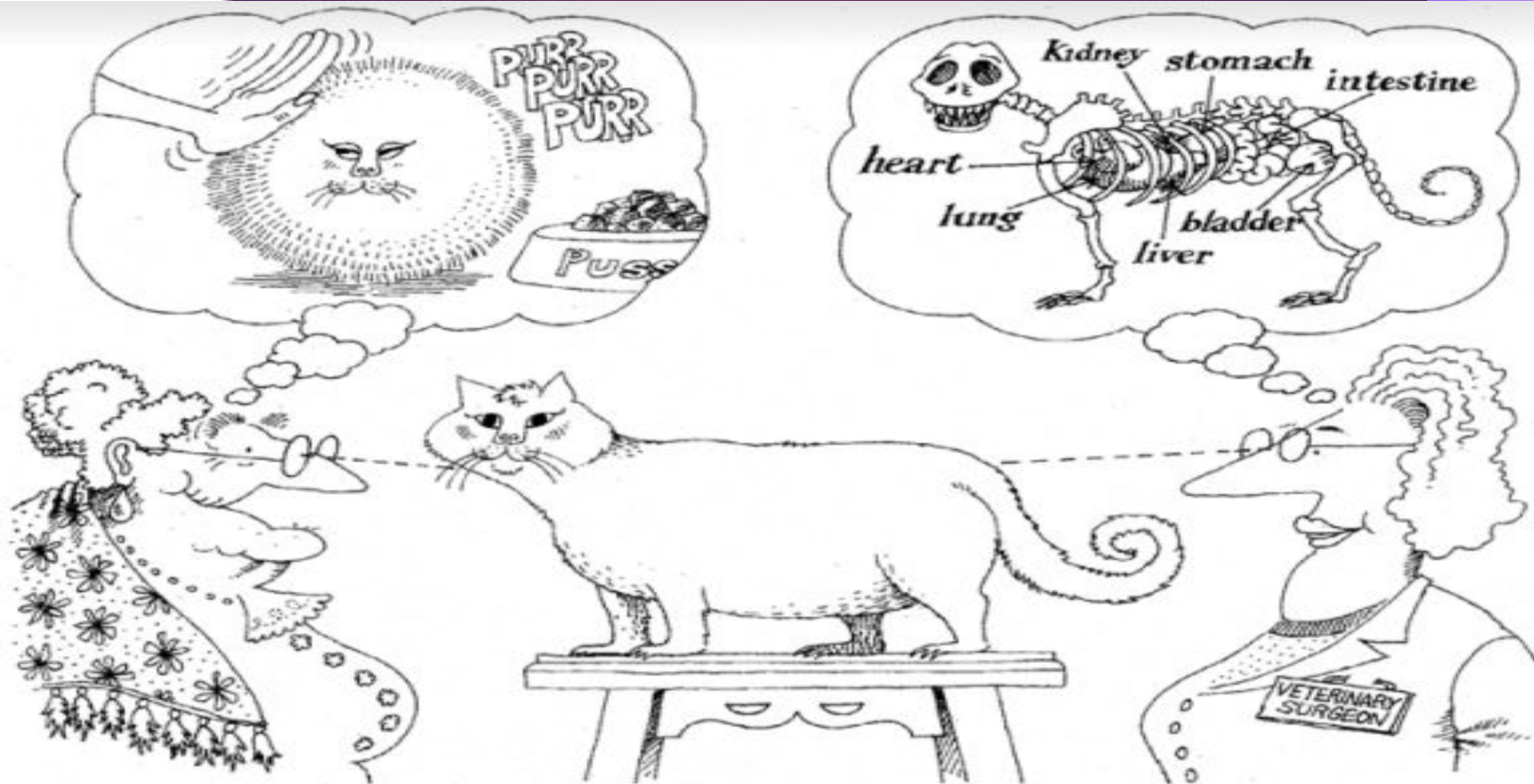


abstraction

An abstraction denotes the essential characteristics of an object that distinguish it from all other kinds of objects and thus provide crisply defined conceptual boundaries, relative to the perspective of the viewer.

An abstraction focuses on the outside view of an object and so serves to separate an object's essential behavior from its implementation.

abstraction



Abstraction focuses on the essential characteristics of some object, relative to the perspective of the viewer.

abstraction

- **Entity abstraction** : An object that represents a useful model of a problem domain or solution domain entity
- **Action abstraction** : An object that provides a generalized set of operations, all of which perform the same kind of function
- **Virtual machine abstraction** : An object that groups operations that are all used by some superior level of control, or operations that all use some junior-level set of operations
- **Coincidental abstraction** : An object that packages a set of operations that have no relation to each other

Examples of Abstraction

Abstraction: Temperature Sensor
Important Characteristics: temperature location
Responsibilities: report current temperature calibrate

Abstraction: Active Temperature Sensor
Important Characteristics: temperature location setpoint
Responsibilities: report current temperature calibrate establish setpoint

encapsulation

- Abstraction and encapsulation are complementary concepts:
- Abstraction focuses on the observable behavior of an object, whereas encapsulation focuses on the implementation that gives rise to this behavior.
- Encapsulation is most often achieved through information hiding (not just data hiding), which is the process of hiding all the secrets of an object typically, the structure of an object is hidden, as well as the implementation of its methods.
- Whereas abstraction “**helps people to think about what they are doing,**” encapsulation “**allows program changes to be reliably made with limited effort**”

encapsulation

Encapsulation is the process of compartmentalizing the elements of an abstraction that constitute its structure and behavior; encapsulation serves to separate the contractual interface of an abstraction and its implementation.

Example - encapsulation

Abstraction: Heater
Important Characteristics: location status
Responsibilities: turn on turn off provide status

Related Candidate Abstractions: Heater Controller, Temperature Sensor

modularity

- The act of partitioning a program into individual components can reduce its complexity to some degree.
- Although partitioning a program is helpful for this reason, a more powerful justification for partitioning a program is that it creates a number of well-defined, documented boundaries within the program.
- These boundaries, or interfaces, are invaluable in the comprehension of the program

modularity



modularity

- Especially for larger applications, in which we may have many hundreds of classes, the use of modules is essential to help manage complexity.
- Modularity is the property of a system that has been decomposed into a set of cohesive and loosely coupled modules.
- cohesive (by grouping logically related abstractions) and
- loosely coupled (by minimizing the dependencies among modules).

hierarchy

- Abstraction is a good thing, but in all except the most trivial applications, we may find many more different abstractions than we can comprehend at one time.
- Encapsulation helps manage this complexity by hiding the inside view of our abstractions.
- Modularity helps also, by giving us a way to cluster logically related abstractions.
- A set of abstractions often forms a hierarchy, and by identifying these hierarchies in design, we greatly simplify our understanding of the problem.



hierarchy

□ **Hierarchy is a ranking or ordering of abstractions.**

□ The two most important hierarchies in a complex system are its class structure (the “is a” hierarchy) and its object structure (the “part of” hierarchy).

Examples - hierarchy

□ Inheritance - is a relationship

Inheritance represents a hierarchy of abstractions, in which a subclass inherits from one or more super classes. Typically, a subclass augments or redefines the existing structure and behavior of its super classes.

For example, a bear “is a” kind of mammal

□ Aggregation - part of relationship

Aggregation is a relationship among classes by which a class can be made up of any combination of objects of other classes.

Example A car has-a motor, car is the whole object or the aggregate, and the motor is a “part-of” the car.

typing

- **“A type is a precise characterization of structural or behavioral properties which a collection of entities all share”**
- Typing is the enforcement of the class of an object, such that objects of different types may not be interchanged, or at the most, they may be interchanged only in very restricted ways.

typing

- **A given programming language may be strongly typed, weakly typed, or even untyped, yet still be called object-oriented.**
- Typing is the enforcement of the class of an object, such that objects of different types may not be interchanged, or at the most, they may be interchanged only in very restricted ways.

typing

There are a number of important benefits to be derived from using strongly typed languages:

- Without type checking, a program in most languages can ‘crash’ in mysterious ways at runtime.
- In most systems, the edit-compile-debug cycle is so tedious that early error detection is indispensable.
- Type declarations help to document programs.
- Most compilers can generate more efficient object code if types are declared..

Examples - typing

- The concepts of strong and weak typing and static and dynamic typing are entirely different.
- Strong and weak typing refers to type consistency, whereas static and dynamic typing refers to the time when names are bound to types.
- Static typing (also known as static binding or early binding) means that the types of all variables and expressions are fixed at the time of compilation
- Dynamic typing (also known as late binding) means that the types of all variables and expressions are not known until runtime.
- A language may be both strongly and statically typed (Ada), strongly typed yet supportive of dynamic typing (C++, Java)

Examples - typing

- Polymorphism is a condition that exists when the features of dynamic typing and inheritance interact.
- Polymorphism represents a concept in type theory in which a single name (such as a variable declaration) may denote objects of many different classes that are related by some common superclass.
- Any object denoted by this name is therefore able to respond to some common set of operations .
- Opposite of polymorphism is monomorphism, which is found in all languages that are both strongly and statically typed.
- Polymorphism is perhaps the most powerful feature of object-oriented programming languages

concurrency

- For certain kinds of problems, an automated system may have to handle many different events simultaneously.
- Other problems may involve so much computation that they exceed the capacity of any single processor.
- In each of these cases, it is natural to consider using a distributed set of computers for the target implementation or to use multitasking.

concurrency

- A single process is the root from which independent dynamic action occurs within a system.
- Every program has at least one thread of control, but a system involving concurrency may have many such threads:
 - Some that are transitory and others that last the entire lifetime of the system's execution.
- Systems executing across multiple CPUs allow for truly concurrent threads of control, whereas systems running on a single CPU can only achieve the illusion of concurrent threads of control, usually by means of some time-slicing algorithm.



concurrency

□ Concurrency is the property that distinguishes an active object from one that is not active.

persistence

- An object in software takes up some amount of space and exists for a particular amount of time.
- Persistence is the property of an object through which its existence transcends time (i.e., the object continues to exist after its creator ceases to exist) and/or space (i.e., the object's location moves from the address space in which it was created).



Benefits of the Object Model

- 1. the use of the object model helps us to exploit the expressive power of object-based and object-oriented programming languages.**
- 2. the use of the object model encourages the reuse not only of software but of entire designs, leading to the creation of reusable application frameworks**
- 3. the use of the object model produces systems that are built on stable intermediate forms, which are more resilient to change. This also means that such systems can be allowed to evolve over time, rather than be abandoned or completely redesigned in response to the first major change in requirements.**

Open Issues

To effectively apply the elements of the object model, we must next address several open issues.

1. What exactly are classes and objects?
2. How does one properly identify the classes and objects that are relevant to a particular application?
3. What is a suitable notation for expressing the design of an object-oriented system?
4. What process can lead us to a well-structured object-oriented system?
5. What are the management implications of using object-oriented design?